

SENAI FATESG

Análise e Desenvolvimento de Sistemas — 3º Semestre
Arquitetura e Projeto de Software

ADR-001

Definição da Stack e Arquitetura Base

Status: Em vigor **Data:** Aula 01 **Autor:** Equipe

Caio Abreu | Cassiano Abreu | Gabriel Naoki | Wyllian Mariano
SENAI FATESG — Goiânia, GO — 01 de abril de 2025

Contexto

O projeto CarRepair precisava de uma base tecnológica sólida para suportar o desenvolvimento de um sistema de gestão de oficina mecânica com alta manutenibilidade e organização didática. Era necessário definir a linguagem, o framework, o banco de dados e o padrão de organização do código antes de iniciar qualquer implementação. Sem essas definições, cada integrante poderia adotar abordagens diferentes, gerando inconsistências e dificuldade de integração.

Decisão

Utilizar Java 21 com Spring Boot no backend, adotando arquitetura de monolito modular com separação obrigatória em camadas: Model, Repository, Validation, Service e Controller. No frontend, adotar Angular 21 com TypeScript. Banco de dados PostgreSQL.

Fluxo obrigatório de todas as requisições:

`Controller → Service → Validation → Repository → Model`

Justificativa

- Java 21 (LTS) traz recursos modernos como Virtual Threads e Pattern Matching, aumentando produtividade e desempenho
- Spring Boot acelera o setup e elimina configuração manual de infraestrutura
- A separação em camadas isola responsabilidades, facilita testes unitários e torna o código mais compreensível para fins acadêmicos
- O monolito modular é adequado para o escopo acadêmico — evita complexidade desnecessária de microsserviços
- Angular 21 com TypeScript garante tipagem estática, organização por componentes e alinhamento com o mercado
- PostgreSQL é robusto, open source, suporta UUID nativo e é amplamente utilizado no mercado

Alternativas Consideradas

- Microsserviços — descartado por adicionar complexidade desnecessária ao escopo acadêmico
- Quarkus ou Micronaut — descartados pela menor familiaridade da equipe com esses frameworks
- MySQL no lugar de PostgreSQL — descartado pela preferência por UUID nativo e recursos avançados do PostgreSQL
- React ou Vue no frontend — descartados em favor do Angular pela estrutura mais rígida e didaticamente mais próxima do padrão enterprise

Consequências

- Todo novo domínio deve obrigatoriamente seguir o fluxo Controller → Service → Validation → Repository → Model
- A separação de responsabilidades aumenta o número de arquivos por domínio, mas garante manutenibilidade
- O backend escalável permite evoluir para microsserviços futuramente sem reescrita total
- A equipe precisa manter disciplina na criação e organização das classes por camada